

Requirement on academic honesty (COMP8090SEF/S209W):

1. Each student must make the following declaration on the cover page of the report

Hong Kong Metropolitan University

COMP S209W/8090SEF
Data Structures, Algorithms And Problem Solving
Data Structures, Algorithms

Course Project Report
Group 10

Submission date:

I declare that:

- (i) I have read and checked that all parts of the project (including proposal, code programs, and reports), they are contributed by me, here submitted is original except for source material explicitly acknowledged;
- (ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration;
- (iii) I am aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;
- (iv) I confirm that I have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and
- (v) I am aware that I should be held responsible and liable to disciplinary actions, irrespective of whether I have signed the declaration and whether I have contributed, directly or indirectly, to the problematic contents.

I confirm that I have read through and understood the project requirements. I understand that failure to comply with the project requirements will result in score deduction.

NAME	SID
NGAN Chi Ho	11552520

2. Each student must acknowledge ALL external resources or code from others referenced. We will only mark the parts that are original, not the parts from others.

1. Introduction: This report studies the heap data structure and the heap sort algorithm. It is commonly used to implement priority queues, where the highest priority or lowest priority element must be accessed efficiently. Heap sort is a comparison-based sorting algorithm that uses the heap structure to sort elements effectively. The purpose of this report is to explain the of heaps, Heap Abstract Data Type, and show how heap sort works in practice. Also discusses the time complexity, practical applications, the advantages and limitations of both the heap and heap sort.

2. Heap Data Structure: A heap is a special type of binary tree that satisfies two conditions which it must be a complete binary tree and satisfy the heap property. A complete binary tree is a binary tree in which all levels are filled except possibly the last level, and the nodes on the last level are filled from left to right. This structural property is important because it allows the heap to be stored efficiently in an array without wasting space. The heap property determines the order between parent nodes and child nodes. There are two main types of heaps:

2.1. Max Heap stored as: [90, 70, 50, 40, 60, 20, 30]: Every parent node is greater than or equal to its children. The root node always contains the largest element in the heap.

2.2. Min Heap stored as: [20, 40, 30, 70, 60, 50, 90]: every parent node is less than or equal to its children, so the root node always contains the smallest element in the heap. It does not store elements in fully sorted order. The heap property guarantees only a local ordering between a parent and its children, rather than a global ordering across the entire structure. Heap is different from a binary search tree, where the ordering rule applies across left and right subtrees. In real world example, in an emergency room triage system, patients can be prioritized by medical urgency so that the most critical cases are handled first. Similarly, a heap can be used in identifying the top 10 trending songs without sorting millions of entries. In graph algorithms such as Dijkstra's algorithm, a min heap is used to efficiently select the next closest node during shortest-path computation. If a node is stored at index i , then: left child = $2i+1$; right child = $2i+2$; parent = $\lfloor (i-1)/2 \rfloor$; This array representation is more memory efficient than using linked tree nodes because no explicit pointers are required. It also makes indexing and implementation simpler.

3. Heap Sort Algorithm: Heap sort is a comparison-based sorting algorithm that uses the heap data structure to sort elements efficiently. The main idea is to first organize the input array into a heap and then repeatedly move the root element into its correct final position in the sorted part of the array. In a max heap, the largest element is always stored at the root. Therefore, by swapping the root with the last unsorted element and then restoring the heap property, the algorithm gradually builds a sorted array in ascending order. This process is repeated until all elements are sorted.

3.1. Python Implementation of heapify() and heap_sort(): The function heapify(arr, n, i) restores the max heap property for the subtree rooted at index i . It begins by assuming that the current node is the largest. It then compares the current

node with its left and right children, located at indices $2i + 1$ and $2i + 2$. If one of the children is larger, the largest index is updated. If the largest value is not already at the root of the subtree, the elements are swapped, and `heapify()` is applied recursively to the affected subtree. So, `heapify()` pushes a smaller value downward until the subtree once again satisfies the max heap property.

3.2. Working Process of `heap_sort(arr)`: The array is first converted into a max heap. This process starts from the last non-leaf node and moves backward to the root. Leaf nodes do not need because they already satisfy the heap property. Second, it repeatedly swaps the root with the last unsorted element. The largest element is therefore moved to the end of the array. The heap size is then reduced and `heapify()` is called again on the root to restore the max heap property in the remaining unsorted part. By repeating these steps, heap sort produces a sorted array from right to left. To perform a Heap Sort on the array `[20, 40, 30, 70, 60, 50]`, we first build a max heap by heapifying from the last non-leaf node, resulting in `[70, 60, 50, 40, 20, 30]`. Next, we swap the root 70 with the last element 30 and heapify the remaining elements to get `[60, 40, 50, 30, 20]`; swap the new root 60 with 20 and heapify to get `[50, 40, 20, 30]`; swap 50 with 30 and heapify to get `[40, 30, 20]`; swap 40 with 20 and heapify to get `[30, 20]`; and finally swap 30 with 20 to leave the last element in place. This iterative process of swapping the maximum element to the end and restoring the heap property results in the final sorted array: `[20, 30, 40, 50, 60, 70]`. The time complexity of `heapify()` is $O(\log n)$ because, in the worst case, an element may need to move down the height of the heap. The overall time complexity of heap sort is $O(n \log n)$, while the space complexity is $O(1)$.

4. Heap Abstract Data Type: It defines the main operations on heap to function as an efficient structure for managing prioritized elements. The key operations are `heapify()`, `insert(x)`, `extractMax()`/ `extractMin()`, `peek()`, and `buildHeap()`.

4.1. `heapify()`: As mentioned before, it is one of the most important procedures in the heap. Its purpose is to restore the heap property for a subtree. If the current node violates the heap property, `heapify()` compares with its children and swaps it with the appropriate child. This process continues until the subtree becomes a valid heap. In a min heap, if the root of a subtree is greater than one of its children, it must be swapped downward until the smaller value reaches the root position. Showing the steps in given: `[50, 40, 30, 70, 60, 20]`; The root node 50 violates the min heap property because it is greater than its children 40 and 30. So, 50 is compared with its children and swapped with the smaller child 30: `[30, 40, 50, 70, 60, 20]`; Then, 50 is compared with its child 20, and swapped again since $50 > 20$ and result in `[30, 40, 20, 70, 60, 50]`. Since this operation may move an element from one level to another, its time complexity is $O(\log n)$.

4.2. `insert(x)`: The operation `insert(x)` adds a new element into the heap. To preserve the complete binary tree structure, the new element is first placed in the next available position, usually at the end of the array. However, this may

violate the heap property. To correct this, the new element is compared with its parent and moved upward through a sift-up process until the heap property is restored. Suppose a max heap is stored as: [20, 40, 30, 70, 60, 50]; If 90 is inserted, it is first appended to the end of the array to maintain the complete binary tree structure: [20, 40, 30, 70, 60, 50, 90]; Then, 90 is compared with its parent 30 and swapped: [20, 40, 90, 70, 60, 50, 30]; Then, 90 is compared with its parent 20 and swapped again result in: [90, 40, 20, 70, 60, 50, 30]; The time complexity of insertion is $O(\log n)$. 4.3. `extractMax()` / `extractMin()`: In a max heap, it removes and returns the root element, which is the largest value, vice versa. To maintain the complete binary tree structure after removal, the last element is moved to the root position. This may violate the heap property, so the new root must be moved downward through a sift-down process until the heap becomes valid again. suppose a min heap is stored as: [20, 40, 30, 70, 60, 50]; The root is 20, which is removed and keep the complete binary tree, the last element 50 is moved to the root: [50, 40, 30, 70, 60]; Then, 50 is compared with its children 40 and 30, and swapped with the smaller child 30; Next, 50 is compared with its children. Since $50 \leq 70$ and $50 \leq 60$, no further swaps are needed: [30, 40, 50, 70, 60]

. Although removing the last element itself is $O(1)$, restoring the heap property through sift-down is $O(\log n)$. Therefore, the overall time complexity of extraction is $O(\log n)$.

4.4. `peek()`: The operation `peek()` returns the root element without removing it. In a max heap, it returns the largest element, vice versa in a min heap. Min Heap out put [20, 40, 30, 70, 60, 50] and Max Heap out put [70, 50, 40, 60, 20, 30]. Since the root is always stored at index 0, this operation is very efficient, with a time complexity of $O(1)$.

4.5. `buildHeap()`: The operation `buildHeap()` transforms an unsorted array into a valid heap. This operation is especially important because it forms the first phase of heap sort. Instead of inserting elements one by one, it processes internal nodes from bottom to top and applies `heapify()` to each node. This bottom-up approach is more efficient and builds the heap in $O(n)$ time. 5 Advantages and Limitations: The heap data offers efficient access to the maximum or minimum element in constant time $O(1)$ as well as efficient insertion and deletion operations in $O(\log n)$. However, the heap does not support efficient searching for arbitrary elements, with worst-case time complexity of $O(n)$, and it does not maintain a globally sorted order. Heap sort is its guaranteed time complexity of $O(n \log n)$ in the best, average, and worst cases. Nevertheless, heap sort has limitations. It is not a stable sorting algorithm, meaning that equal elements may not retain their original relative order after sorting. 6. Conclusion: The heap is an efficient data structure for managing prioritized elements, while heap sort is a powerful to built directly upon the heap property. The heap supports fast access to the largest or smallest element, insertion and deletion. Heap sort uses these properties to repeatedly place elements into their correct positions, producing a sorted array in $O(n \log n)$ time.

Reference:

Heap data

<https://www.geeksforgeeks.org/dsa/heap-data-structure/>

Heap Sort

<https://www.geeksforgeeks.org/dsa/heap-sort/>

Appendix A

2. Array list

Index [0, 1, 2, 3, 4, 5, 6]

List: [90, 70, 50, 40, 60, 20, 30]

2. Max heap

Stored as: [90, 70, 50, 40, 60, 20, 30]

90 is the root at index 0

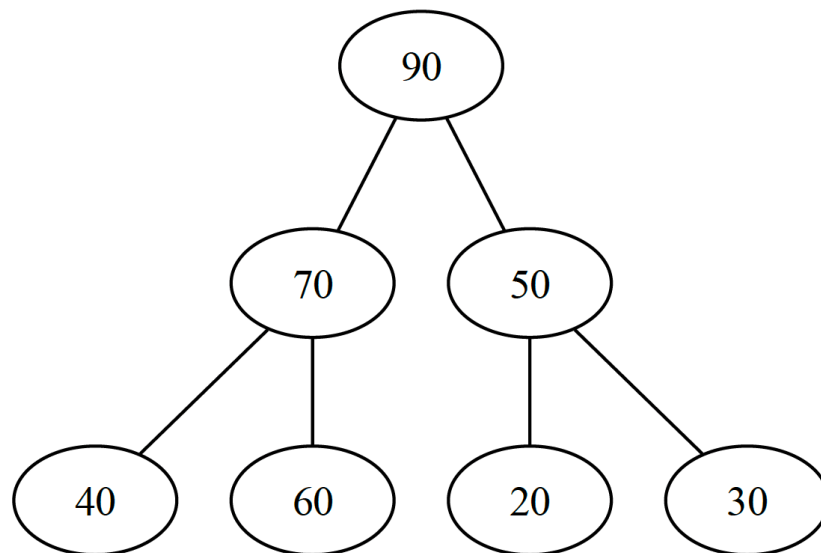
70 is the left child of 90 at index 1

50 is the right child of 90 at index 2

40 and 60 are the children of 70

20 and 30 are the children of 50

Graph of a Max Heap



This is a max heap because each parent is greater than or equal to its children.

2.2 Min heap

Stored as: [20, 40, 30, 70, 60, 50, 90]

20 is the root at index 0

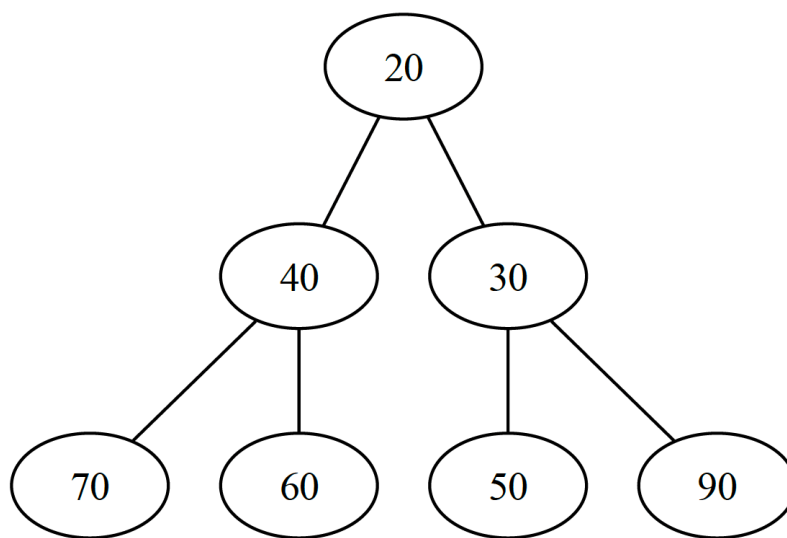
40 is the left child of 20 at index 1

30 is the right child of 20 at index 2

70 and 60 are the children of 40

50 and 90 are the children of 30

2.2 Example of a Min Heap



This is a min heap because each parent is less than or equal to its children.

Array representation

left child = $2i + 1$

right child = $2i + 2$

parent = $(i - 1) // 2$

index [0, 1, 2, 3, 4, 5, 6]

list : [20, 40, 30, 70, 60, 50, 90]

For the element 40 at index

Left Child: $2i+1=2(1)+1=3$, The value at array index 3 is 70.

Right Child: $2i+2=2(1)+2=4$, The value at array index 4 is 60.

Parent: $(1 - 1) // 2=0$, The value at array index 0 is 20.

3.Heap Sort

Max heap Example

Input array:

[20, 40, 30, 70, 60, 50]

Heap Sort Concept

Step 1: Build a max heap

Step 2: Swap root with last unsorted element

Step 3: Reduce heap size

Step 4: Heapify root

Step 5: Repeat until sorted

	Index	0	1	2	3	4	5
Original array	Value	20	40	30	70	60	50
After build max heap	Value	70	60	50	40	20	30
After 1st swap	Value	30	60	50	40	20	[70]
After heapify	Value	60	40	50	30	20	[70]
After 2nd swap	Value	20	40	50	30	[60]	[70]
After heapify	Value	50	40	20	30	[60]	[70]
After 3rd swap	Value	30	40	20	[50]	[60]	[70]
After heapify	Value	40	30	20	[50]	[60]	[70]
After 4th swap	Value	20	30	[40]	[50]	[60]	[70]
After heapify	Value	30	20	[40]	[50]	[60]	[70]
After 5th swap	Value	20	[30]	[40]	[50]	[60]	[70]

Iteration 1:

Swap 70 and 30: [30, 60, 50, 40, 20 | 70]

Heapify root 30: [60, 40, 50, 30, 20 | 70]

Iteration 2:

Swap 60 and 20: [20, 40, 50, 30 | 60, 70]

Heapify root 20: [50, 40, 20, 30 | 60, 70]

Iteration 3:

Swap 50 and 30: [30, 40, 20 | 50, 60, 70]

Heapify root 30: [40, 30, 20 | 50, 60, 70]

Iteration 4:

Swap 40 and 20: [20, 30 | 40, 50, 60, 70]

Heapify root 20: [30, 20 | 40, 50, 60, 70]

Iteration 5:

Swap 30 and 20: [20 | 30, 40, 50, 60, 70]

Final Sorted Array: [20, 30, 40, 50, 60, 70]

3.1 Python Implementation of Heap Sort

The following code is the heap sort algorithm:

```

3.1
def heapify(arr, n, i): # Time Complexity: O(log n)
    # Assume the current index i is the largest
    largest = i
    # Calculate the index of left child
    left = 2 * i + 1
    # Calculate the index of right child
    right = 2 * i + 2
    # Check if left child exists and is greater than current largest
    if left < n and arr[left] > arr[largest]:
        # Update largest to left child index
        largest = left
    # Check if right child exists and is greater than current largest
    if right < n and arr[right] > arr[largest]:
        # Update largest to right child index
        largest = right
    # If the largest value is not the root
    if largest != i:
        # Swap the root with the largest child
        print(f'Swap: {arr[i]} with {arr[largest]}')
        arr[i], arr[largest] = arr[largest], arr[i]
        print(f'Current array: {arr}')
        # Recursively apply heapify to the affected subtree
        heapify(arr, n, largest)
arr = [20, 40, 30, 70, 60, 50]
print("Original array:", arr)
heapify(arr, len(arr), 2) # Heapify the subtree rooted at index 2, # 30 swapped with 50
heapify(arr, len(arr), 1) # Heapify the subtree rooted at index 1, # 40 swapped with 70
heapify(arr, len(arr), 0) # Heapify the subtree rooted at index 0, # 20 swapped with 70,
                           # then 20 swapped with 60

#3.2
def heap_sort(arr):
    # Get the length of the array
    n = len(arr)
    #First phase

```



```
# Build a max heap from the array
# Start from the last non-leaf node and move upward
for i in range(n // 2 - 1, -1, -1): # Time Complexity: O(n) n
    # Apply heapify to each node
    heapify(arr, n, i)
#Second phase
# Extract elements from the heap one by one
for i in range(n - 1, 0, -1): # Time Complexity: O(n log n)
    print(f'Before moving root to end (i={i}):', arr) # Debug print
    # Move current root (largest element) to the end
    arr[i], arr[0] = arr[0], arr[i]
    print(f'After moving root to end (i={i}):', arr) # Debug print
    # Restore heap property on the reduced heap
    heapify(arr, i, 0)
arr = [20, 40, 30, 70, 60, 50]
heap_sort(arr)
print(arr) # Output: [20, 30, 40, 50, 60, 70]
```

Output:

Original array: [20, 40, 30, 70, 60, 50]

Swap: 30 with 50

Current array: [20, 40, 50, 70, 60, 30]

Swap: 40 with 70

Current array: [20, 70, 50, 40, 60, 30]

Swap: 20 with 70

Current array: [70, 20, 50, 40, 60, 30]

Swap: 20 with 60

Current array: [70, 60, 50, 40, 20, 30]

Swap: 30 with 50

Current array: [20, 40, 50, 70, 60, 30]

Swap: 40 with 70

Current array: [20, 70, 50, 40, 60, 30]

Swap: 20 with 70

Current array: [70, 20, 50, 40, 60, 30]

Swap: 20 with 60

Current array: [70, 60, 50, 40, 20, 30]

Before moving root to end (i=5): [70, 60, 50, 40, 20, 30]

After moving root to end (i=5): [30, 60, 50, 40, 20, 70]

Swap: 30 with 60

Current array: [60, 30, 50, 40, 20, 70]

Swap: 30 with 40

Current array: [60, 40, 50, 30, 20, 70]

Before moving root to end (i=4): [60, 40, 50, 30, 20, 70]

After moving root to end (i=4): [20, 40, 50, 30, 60, 70]

Swap: 20 with 50

Current array: [50, 40, 20, 30, 60, 70]

Before moving root to end (i=3): [50, 40, 20, 30, 60, 70]

After moving root to end (i=3): [30, 40, 20, 50, 60, 70]

Swap: 30 with 40

Current array: [40, 30, 20, 50, 60, 70]

Before moving root to end (i=2): [40, 30, 20, 50, 60, 70]

After moving root to end (i=2): [20, 30, 40, 50, 60, 70]

Swap: 20 with 30

Current array: [30, 20, 40, 50, 60, 70]

Before moving root to end (i=1): [30, 20, 40, 50, 60, 70]

After moving root to end (i=1): [20, 30, 40, 50, 60, 70]

[20, 30, 40, 50, 60, 70]

4.1 min-heap

```
def min_heapify(arr, n, i):
    # Time Complexity: O(log n)
    smallest = i
    left = 2 * i + 1
    right = 2 * i + 2
    # Check if left child exists and is smaller than the current smallest
    if left < n and arr[left] < arr[smallest]:
        smallest = left
    # Check if right child exists and is smaller than the current smallest
    if right < n and arr[right] < arr[smallest]:
        smallest = right
    # If the smallest value is not the root, perform a swap
    if smallest != i:
        print(f'Swap: {arr[i]} with {arr[smallest]}')
        arr[i], arr[smallest] = arr[smallest], arr[i]
        print(f'Current array: {arr}')
    # Recursively apply min_heapify to the affected subtree
```

```

        min_heapify(arr, n, smallest)
arr = [50, 40, 30, 70, 60, 20]
print("Original array:", arr)
min_heapify(arr, len(arr), 0)
print("Final Result:", arr)

```

Out put:

Original array: [50, 40, 30, 70, 60, 20]

Swap: 50 with 30

Current array: [30, 40, 50, 70, 60, 20]

Swap: 50 with 20

Current array: [30, 40, 20, 70, 60, 50]

Final Result: [30, 40, 20, 70, 60, 50]

4.2 insert(x) – Example (Max Heap)

```

def insert(heap, val):
    # Step 1: Append the new element to the end of the array
    # Time Complexity: O(1)
    heap.append(val)
    print(f"Append {val}: {heap}")
    # Step 2: Sift-up to restore the Max Heap property
    # Time Complexity: O(log n)
    current_idx = len(heap) - 1
    while current_idx > 0:
        parent_idx = (current_idx - 1) // 2
        # If the new element is greater than its parent, swap them
        if heap[current_idx] > heap[parent_idx]:
            print(f"Swap: {heap[current_idx]} with parent {heap[parent_idx]}")
            heap[current_idx], heap[parent_idx] = heap[parent_idx],
            heap[current_idx]
            print(f"Current array: {heap}")
            # Move up to the parent's index
            current_idx = parent_idx
        else:
            # Heap property is satisfied
            break
    # Example from the image
    max_heap = [20, 40, 30, 70, 60, 50]
    print("Original Max Heap:", max_heap)

```

```
insert(max_heap, 90)
print("Final Max Heap after insertion:", max_heap)
```

Out put:

Original Max Heap: [20, 40, 30, 70, 60, 50]

Append 90: [20, 40, 30, 70, 60, 50, 90]

Swap: 90 with parent 30

Current array: [20, 40, 90, 70, 60, 50, 30]

Swap: 90 with parent 20

Current array: [90, 40, 20, 70, 60, 50, 30]

Final Max Heap after insertion: [90, 40, 20, 70, 60, 50, 30]

4.3 extractMin() – Example (Min Heap)

```
def min_heapify(arr, n, i):
    # Standard sift-down operation
    # Time Complexity: O(log n)
    smallest = i
    left = 2 * i + 1
    right = 2 * i + 2
    if left < n and arr[left] < arr[smallest]:
        smallest = left
    if right < n and arr[right] < arr[smallest]:
        smallest = right
    if smallest != i:
        print(f"Sift-down: Swap {arr[i]} with {arr[smallest]}")
        arr[i], arr[smallest] = arr[smallest], arr[i]
        min_heapify(arr, n, smallest)

def extract_min(heap):
    if not heap:
        return None
    # If there's only one element
    if len(heap) == 1:
        return heap.pop()
    # 1. Store the root (minimum element) to return later
    min_val = heap[0]
    print(f"Extracting Min (Root): {min_val}")
    # 2. Move the last element to the root position
    last_element = heap.pop()
```

```

    heap[0] = last_element
    print(f"Moved last element {last_element} to root: {heap}")
    # 3. Restore heap property using sift-down (min_heapify)
    # Time Complexity: O(log n)
    min_heapify(heap, len(heap), 0)
    return min_val
# Example from the image
min_heap = [20, 40, 30, 70, 60, 50]
print("Initial Min Heap:", min_heap)
removed = extract_min(min_heap)
print(f"Removed value: {removed}")
print("Final Min Heap:", min_heap)

```

Output:

Initial Min Heap: [20, 40, 30, 70, 60, 50]
 Extracting Min (Root): 20
 Moved last element 50 to root: [50, 40, 30, 70, 60]
 Sift-down: Swap 50 with 30
 Removed value: 20
 Final Min Heap: [30, 40, 50, 70, 60]

4.4 peek()

```

def peek(heap):
    # Time Complexity: O(1)
    # Since the root is always at index 0, we can access it instantly.

    if not heap:
        return None # Or raise an error if the heap is empty

    return heap[0]
# --- Min Heap Example ---
min_heap = [20, 40, 30, 70, 60, 50]
print(f"Min Heap: {min_heap}")
print(f"Peek (Smallest): {peek(min_heap)}") # Output: 20
# --- Max Heap Example ---
max_heap = [70, 50, 40, 60, 20, 30]
print(f"Max Heap: {max_heap}")
print(f"Peek (Largest): {peek(max_heap)}") # Output: 70

```

Output

Min Heap: [20, 40, 30, 70, 60, 50]

Peek (Smallest): 20

Max Heap: [70, 50, 40, 60, 20, 30]

Peek (Largest): 70

4.5 buildHeap() – Example (Min Heap)

```
def min_heapify(arr, n, i):
    # Time Complexity: O(log n)
    smallest = i
    left = 2 * i + 1
    right = 2 * i + 2
    # Check if left child exists and is smaller than root
    if left < n and arr[left] < arr[smallest]:
        smallest = left
    # Check if right child exists and is smaller than the current smallest
    if right < n and arr[right] < arr[smallest]:
        smallest = right
    # If smallest is not root
    if smallest != i:
        print(f"Swap: {arr[i]} with {arr[smallest]}")
        arr[i], arr[smallest] = arr[smallest], arr[i]
        # Recursively heapify the affected sub-tree
        min_heapify(arr, n, smallest)

def build_heap(arr):
    # Time Complexity: O(n)
    n = len(arr)
    # Start from the last internal node (n//2 - 1) and move upward to the root
    # For a list of 6 elements, n//2 - 1 is index 2
    for i in range(n // 2 - 1, -1, -1):
        print(f"Processing node at index {i} (value {arr[i]}):")
        min_heapify(arr, n, i)
        print(f"Array state: {arr}")
    # Example from the image
    arr = [50, 40, 30, 70, 60, 20]
    print("Initial unsorted array:", arr)
    build_heap(arr)
```

```
print("Final Min Heap:", arr)
```

Out put:

Initial unsorted array: [50, 40, 30, 70, 60, 20]

Processing node at index 2 (value 30):

Swap: 30 with 20

Array state: [50, 40, 20, 70, 60, 30]

Processing node at index 1 (value 40):

Array state: [50, 40, 20, 70, 60, 30]

Processing node at index 0 (value 50):

Swap: 50 with 20

Swap: 50 with 30

Array state: [20, 40, 30, 70, 60, 50]

Final Min Heap: [20, 40, 30, 70, 60, 50]
